

Graph Convolutional Networks (GCN)

Introduction

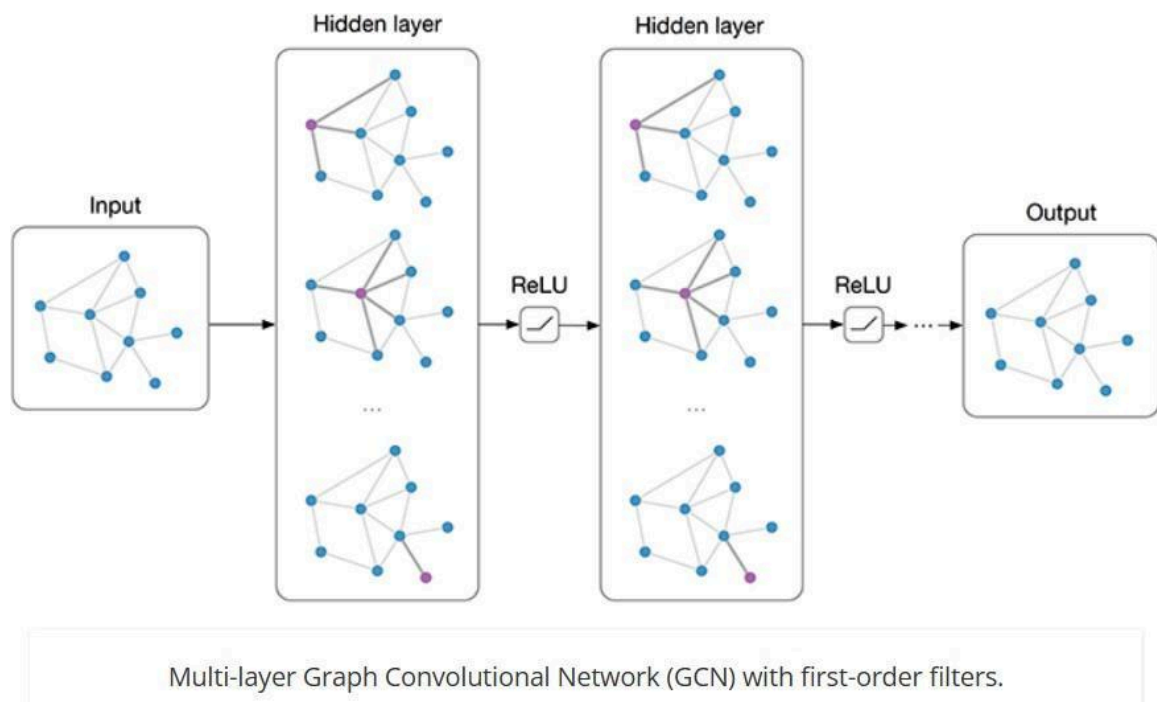
Graph Convolutional Networks are neural networks designed to work on graph data. In a graph, information is stored in nodes and connections (edges) between them. GCNs are important because many problems use graph data (for example, social networks or molecules), and GCNs can learn from the relationships in the graph.

Definition

A GCN is a type of neural network that learns from graph-structured data by combining information from each node and its neighbors.

Basic Concept

The main idea is **neighbour aggregation**. Each node updates itself by using its own data and the data of connected neighbours. A simple analogy: Imagine students in a **Southeast University study group** preparing for the CSE final exam. Each student (node) learns not only from their own notes but also from friends (neighbors) who understand different topics. If one student mastered "Graph Theory" and another mastered "Neural Networks," they share knowledge through the group, and everyone ends up with a better understanding of the entire syllabus.

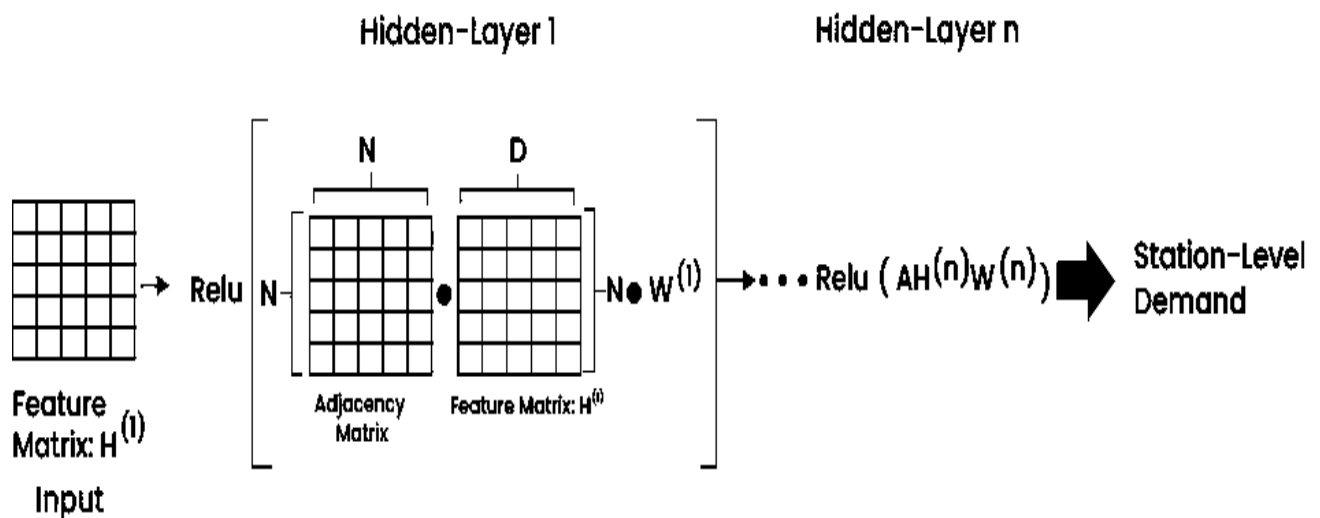


This diagram shows how a Graph Convolutional Network processes data through multiple layers. At the input stage, each node has initial features. In the hidden layers, nodes update their features by combining information from their neighboring nodes. After each layer, an

activation function such as ReLU is applied to introduce non-linearity. As the data passes through multiple layers, the model captures more complex relationships between nodes. Finally, the output layer produces the updated node representations, which can be used for tasks like classification or prediction.

Architecture of GCNs

GCNs typically consist of multiple layers, each responsible for refining node embeddings by aggregating information from neighbors at increasing distances. The layers are:



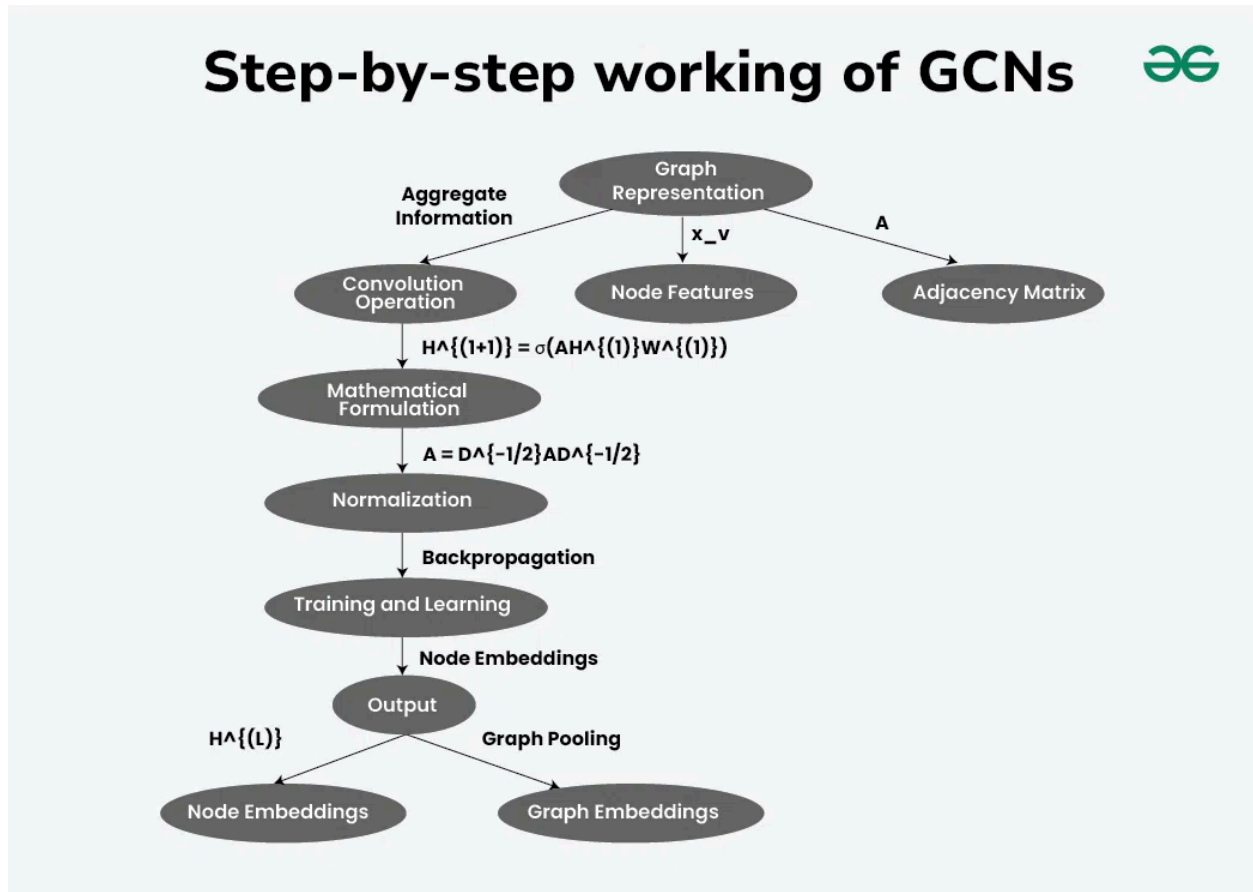
1. **Input Layer:** The input layer initializes the node features, usually from raw data or pre-trained embeddings.
2. **Hidden Layers:** Hidden layers perform the graph convolution operations, progressively aggregating and transforming node features.

Graph Convolutional Layers: These layers perform the convolution operation on the graph. Each layer updates the feature representation of a node by aggregating the features of its neighbors.

Activation Functions: Non-linear functions such as ReLU are applied to the output of each convolutional layer to introduce non-linearity into the model.

Pooling Layers: These layers reduce the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures.

3. **Output Layer:** The output layer produces the final node embeddings or predictions, depending on the task (e.g., node classification, link prediction).
4. **Fully Connected Layers:** These layers are used at the end of the network to perform tasks such as classification or regression.



GCN Important Formulas

1. Graph Representation

$$G = (V, E)$$

2. Adjacency Matrix

$$A \in R^{N \times N}$$

Where:

$$A_{ij} = \begin{cases} 1, & \text{if node } i \text{ and } j \text{ are connected} \\ 0, & \text{otherwise} \end{cases}$$

3. Add Self-Loops

$$\tilde{A} = A + I$$

4. Degree Matrix

$$D_{ii} = \sum_j A_{ij}$$

After adding Self-loop :

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$

5. Symmetric Normalization

$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$$

6. GCN Layer / Propagation Rule

most important formula:

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)})$$

or full form:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right)$$

7. Initial Node Features

$$H^{(0)} = X$$

8. First GCN Layer

$$H^{(1)} = \sigma(\hat{A} X W^{(0)})$$

9. Second GCN Layer

$$H^{(2)} = \sigma(\hat{A} H^{(1)} W^{(1)})$$

10. Node-wise Message Aggregation

For one node i :-

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in N(i) \cup \{i\}} \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}} h_j^{(l)} W^{(l)}\right)$$

11. ReLU Activation

$$\text{ReLU}(x) = \max(0, x)$$

Complete GCN Flow

$$G = (V, E) \rightarrow A \rightarrow \tilde{A} = A + I \rightarrow \tilde{D} \rightarrow \hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} \rightarrow H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)})$$

Most important 5 formulas:

$$G = (V, E)$$

$$\tilde{A} = A + I$$

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)})$$

Example

Social network friend suggestion: Nodes are people, edges are friendships. Each person has features (like interests). A GCN can predict new friends by looking at a person's friends and friends-of-friends.



The network learns that people with similar friend circles often become friends themselves

Real-Life Example: Dhaka Traffic Prediction

Context: Dhaka is one of the most congested cities in the world. Researchers from **Independent University Bangladesh (IUB)** have applied GCNs to predict traffic density using data from the Bangladesh Road Transport Authority (BRTA)

How it works:

1. **Nodes:** Road intersections (Gulshan, Mohakhali, Farmgate, Shahbagh, etc.)
2. **Edges:** Roads connecting intersections
3. **Features:** Vehicle density, average speed, lane occupancy from CCTV cameras
4. **Goal:** Predict traffic congestion 1 hour ahead

Why we need this:

Time Savings: Dhaka residents lose 3.2 hours daily to traffic [General knowledge]

Fuel Efficiency: Reduce wastage of expensive petrol/diesel

Emergency Response: Ambulances can take optimal routes

Economic Impact: Bangladesh loses \$6.5 billion annually due to traffic congestion

How we use it:

- BRTA installs sensors at major intersections
- Data feeds into GCN model running on cloud servers
- Predictions displayed on digital billboards and mobile apps (like "Dhaka Traffic" app)
- Traffic police adjust signal timings proactively

Results from IUB Research :

- GCN achieved RMSE of 9.10 and MAE of 8.00
- Gated Graph Neural Networks (GGNNs) performed even better with MAE of 7.1
- The model successfully predicts congestion at Farmgate during rush hour by learning from patterns at nearby Shahbagh and Science Lab intersections

Real-Life Applications

❖ **Social network analysis and friend recommendation.**

The Problem:

How does Facebook or LinkedIn suggest "**People You May Know**"? How

GCN Solves It:

Nodes: Users

Edges: Friendships, follows, or interactions

Features: User profile data, interests, activity

What the GCN learns: "People with similar friend circles often become friends themselves" (homophily)

Real Example:

If Person A is friends with B and C, and B is friends with C and D, the GCN predicts A and D are likely to connect — even if they have never interacted directly. The network learns from friendship triangles.

Why regular ML fails:

A standard classifier sees users as isolated rows in a spreadsheet. It misses the fact that "your friends' friends" is a powerful predictor.

❖ **Fraud detection in financial networks.**

The Problem:

Scammers create networks of fake bKash accounts. One account receives money from 50 victims, then transfers to a "cash-out" agent in Dhaka. How do you catch the ring?

Real Setup:

- Nodes: bKash accounts (users, agents, merchants)
- Edges: Transactions (Send Money, Cash Out, Payment)
- Features: Transaction amount, frequency, time-of-day, device ID, location (tower data)
- What GCN Predicts: *"This account received ₳5,000 from 20 new accounts in 1 hour, then cashed out at a single agent in Gabtoli. This is a fraud ring — freeze the agent account."*

Why This Matters for Bangladesh:

- Bangladesh leads the world in mobile money density. bKash has 60+ million accounts.
- Standard rule-based systems flag large transactions. GCNs flag network structures — like a hub-and-spoke pattern of scamming that no single transaction reveals.

❖ **Predicting molecule properties in chemistry (atoms as nodes).**

Flood Prediction — Jamuna-Meghna-Brahmaputra Basin

The Problem:

Every monsoon, Bangladesh floods. But which villages will flood first? And which relief centers should prepare?

Real Setup:

Nodes: River monitoring stations (Bahadurabad, Hardinge Bridge), villages (chars in Sirajganj, Kurigram), flood shelters, district offices

Edges: River connections (Jamuna flows into Meghna), road connections (which villages connect to which highways), embankment adjacency

Features: Water level (from BWDB sensors), rainfall (from BMD stations), soil saturation, embankment height

What GCN Predicts: "If water level at Bahadurabad crosses danger level, these 5 downstream villages in Sirajganj will flood in 18 hours. Pre-position boats at these 2 shelters."

Why This Matters for Bangladesh:

BWDB (Bangladesh Water Development Board) has 200+ monitoring stations. But they currently analyze each station independently.

A GCN understands that Sirajganj flooding depends not just on local rain, but on upstream water release from India. The graph encodes that "neighbor" relationship.

- **Knowledge graph reasoning (entities and relations).**
- **Traffic prediction in road networks.**

Advantages	Disadvantages
Handles Irregular Data — Works naturally with messy, non-grid structures like Dhaka's road network. No need for perfectly organized data.	Scalability Issues — Extremely large graphs (e.g., all roads in Bangladesh) require sampling or chunking to process efficiently
Learns Hidden Relationships — Captures patterns across connected nodes. For example, if Mohakhali is jammed, the model learns that Gulshan will likely jam 15 minutes later.	Black-Box Nature — Difficult to explain why the model makes a specific prediction (e.g., why it predicted jam at Farmgate).
Scales with Growth — Can handle thousands of intersections and expands as the city network grows.	Needs Retraining — When the physical network changes (new flyovers, road closures), the model must be updated or predictions become outdated.

Training Graph Convolutional Networks (GCNs) Loss Functions

- **Cross-Entropy Loss:** For classification (e.g., "congested" vs "not congested")
- **Mean Squared Error:** For regression (predicting exact traffic volume)

Optimization

- **Adam Optimizer:** Learning rate starts at 0.001, drops to 0.0001 after 50 epochs
- **Backpropagation:** Gradients flow through the graph structure

Regularization

- **Dropout:** Randomly set 50% of features to zero during training (prevents overfitting to specific Dhaka roads)
- **Weight Decay:** Penalizes large weights to keep model simple

For GCNs: Graph Convolutional Networks extend neural networks to connected data, making them essential for solving Bangladesh's traffic crisis, optimizing agricultural supply chains, and improving social services. By learning from relationships between data points, GCNs provide insights that traditional methods miss.